ASSIGNMENT NO. 11

PROBLEM STATEMENT:

Write a program in C to accept a number from the user and compute the following:

a) Square root of the number

b) Square of the number

c) Cube of the number

d) Check whether the number is prime

e) Factorial of the number

f) Prime factors of the number

OBJECTIVE:

• To understand the use of mathematical operations in C.

• To implement loops and conditional statements

. • To perform number-based computations.

• To enhance logical thinking using menu-driven programs.

THEORY:

Numbers and their mathematical properties play a vital role in programming. Operations such as square, cube, and square root are basic mathematical computations. Prime number checking involves verifying whether a number has exactly two divisors. Factorial is calculated by multiplying a sequence of integers. Prime factorization breaks a number into its prime components. In C programming, these operations can be efficiently implemented using loops, conditional statements, and mathematical functions.

PLATFORM:

Linux

CODE:

```c
#include <stdio.h>
#include <math.h>

unsigned long long factorial(int n) {
    if (n < 0) return 0;
    if (n == 0 || n == 1) return 1;
    unsigned long long fact = 1;
    for (int i = 2; i <= n; i++) {
        fact *= i;
    }
    return fact;
}

int is_prime(int n) {
    if (n <= 1) return 0;
    if (n <= 3) return 1;
    if (n % 2 == 0 || n % 3 == 0) return 0;
    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return 0;
    }
    return 1;
}

void prime_factors(int n) {
    if (n <= 1) {
        printf("No prime factors\n");
        return;
    }
    printf("Prime factors: ");
    // Check for 2
    while (n % 2 == 0) {
        printf("2 ");
        n /= 2;
    }
    // Check odd factors
    for (int i = 3; i * i <= n; i += 2) {
        while (n % i == 0) {
            printf("%d ", i);
            n /= i;
        }
```

```
    }
    // Check odd factors
    for (int i = 3; i * i <= n; i += 2) {
        while (n % i == 0) {
            printf("%d ", i);
            n /= i;
        }
    }
    if (n > 1) printf("%d ", n);
    printf("\n");
}

int main() {
    double num;
    printf("Enter a number: ");
    scanf("%lf", &num);

    int n = (int)num;

    // a) Square root
    printf("Square root: ");
    if (num >= 0) {
        printf("%.2f\n", sqrt(num));
    } else {
        printf("Not real\n");
    }

    // b) Square
    printf("Square: %.2f\n", num * num);

    // c) Cube
    printf("Cube: %.2f\n", num * num * num);

    // d) Prime check (for integer part)
    if (num == n) {
        if (is_prime(n)) {
            printf("Prime: Yes\n");
        } else {
            printf("Prime: No\n");
        }
```

```c
    // d) Prime check (for integer part)
    if (num == n) {
        if (is_prime(n)) {
            printf("Prime: Yes\n");
        } else {
            printf("Prime: No\n");
        }
    } else {
        printf("Prime: N/A (non-integer)\n");
    }

    // e) Factorial (for integer part if non-negative)
    if (num == n && n >= 0) {
        printf("Factorial: %llu\n", factorial(n));
    } else {
        printf("Factorial: N/A\n");
    }

    // f) Prime factors (for positive integer part)
    if (num == n && n > 0) {
        prime_factors(n);
    } else {
        printf("Prime factors: N/A\n");
    }

    return 0;
}
```

OUTPUT:

```
Enter a number: 2
Square root: 1.41
Square: 4.00
Cube: 8.00
Prime: Yes
Factorial: 2
Prime factors: 2
```

CONCLUSION:

Thus, the program to perform various mathematical operations on a number was successfully implemented using C programming. This program demonstrates the use of loops, conditional statements, switch-case, and mathematical functions.